

Dynamic Model Switching for Vision-Based Inspection in UAV Applications

Ganapathi Teja Chirravuri

Department of Computer Engineering
Chemnitz University of Technology
Chemnitz, Germany
tejachirravuri@gmail.com

Batbayar Battseren

Department of Computer Engineering
Chemnitz University of Technology
Chemnitz, Germany
batbayar.battseren@informatik.tu-chemnitz.de

Wolfram Hardt

Department of Computer Engineering
Chemnitz University of Technology
Chemnitz, Germany
dewolfram.hardt@informatik.tu-chemnitz.de

Abstract—UAV inspection videos contain both easy and difficult frames. A small detector is usually fast, but it can miss objects in cluttered or occluded views. A larger detector is more reliable, but it costs more time and energy. This paper studies dynamic model switching (DMS): for each frame, a controller chooses whether to use the fast or the accurate detector. The goal is not only to switch models, but to check whether switching is useful. We evaluate eight policies, including static baselines, scene-feature triggers, hysteresis controllers, a multi-proxy test, and a fast-detector confidence watchdog. Runtime accounting includes the controller and proxy costs, not only detector latency. A policy can look good simply because it often uses the accurate detector. To avoid this, we compare each policy with random switching at the same accurate-detector usage. The difference is called informed gain. The framework is evaluated on 14 UAV insulator inspection videos across glass and porcelain domains, producing 84 completed runs and 672 CPU/CUDA policy rows. Separate Jetson Nano TensorRT FP16 measurements provide detector-level energy evidence, and a secondary Anti-UAV control benchmark uses 6 208 ground-truth images. On the CPU common nano-to-large slice, the confidence-watchdog policy closes 43 % of the runtime-quality gap at 66 % of accurate-only runtime, while local-contrast hysteresis gives the strongest agreement-oriented operating point. A multi-proxy EMA-deviation policy is a reproducible negative result with informed gain below zero. The results show that DMS is a conditional runtime-quality tradeoff rather than a hardware-independent acceleration result.

Index Terms—dynamic inference, model switching, UAV inspection, power line inspection, insulator detection, object detection, edge computing, embedded systems, energy efficiency, informed gain

I. INTRODUCTION

UAV visual inspection of power-line corridors is a common application for YOLO-family object detectors [1]–[4]. Most systems use one detector for the whole video. This is simple, but it forces the same computation on every frame. Some frames are easy: the object is large, clear, and isolated. Other frames are hard: the object is small, partly hidden, or mixed with towers, wires, and background clutter. A fast detector is attractive on easy frames, while a larger detector may be worth the cost on hard frames. This paper asks whether a UAV inspection system can make that choice per frame, and whether the choice is better than chance after runtime costs are included.

Dynamic model switching (DMS) is the idea of making this per-frame choice automatically. Closely related lines of work

include adaptive inference inside a single network — early-exit networks [5], conditional computation, and sub-network selection at deployment time [6]; surveys are available in [7], [8]. Edge-oriented model switching has appeared more recently, with EdgeMLBalancer [9] triggering on CPU utilisation rather than on visual content. What is missing from these lines is a UAV-inspection setting in which (i) the trigger reads visual content or fast-detector confidence, (ii) the controller’s runtime cost is billed, (iii) multiple hardware tiers including embedded are covered, and (iv) the evaluation checks whether the trigger is better than random switching with the same accurate-detector usage.

Our contribution is fourfold. First, we describe a runtime-aware DMS framework whose per-frame decision is interpretable and reproducible (Section III). Second, we introduce a random-baseline-corrected informed-gain metric that isolates the routing skill of a trigger from the share of frames routed to the accurate detector (Section III-C). Third, we report a multi-domain, multi-platform evaluation across 672 policy-rows, together with a separately controlled benchmark on Anti-UAV [10] (Sections IV and V). Fourth, we disclose two negative results that constrain the applicability of the framework: an EMA-deviation multi-proxy policy that fails the informed-gain test, and a hardware-dependent result in which a high-end GPU reduces the detector gap to a point at which scene-proxy switching is no longer economical (Sections V-D and V-A).

II. RELATED WORK

a) Adaptive inference and conditional computation: Early-exit networks like BranchyNet [5] attach intermediate classifiers and stop forward execution as soon as confidence crosses a threshold. Once-for-All [6] trains a super-network and selects a specialised sub-network per device at deployment. Both modify a single network internally rather than switching between separate detector pairs, so deployed-runtime accounting is implicit and there is no notion of routing matched-random baseline.

b) Edge model switching: EdgeMLBalancer [9] is a closely related recent work on runtime model switching. Its trigger is system-state (CPU utilisation), and the reported metric is throughput rather than informed gain over a matched-

random baseline. This is sufficient for load-shedding but does not establish that the trigger contains frame-level signal.

c) *UAV inspection*: Recent work deploys YOLO variants with DeepStream pipelines on Jetson Nano for transmission-line inspection [3], [4]. Such deployments fix the model before flight and do not consider per-frame switching.

d) *Embedded perception*: A reference architecture for real-time MAV perception is provided in [11]. The present work follows the same systems-oriented view, but focuses specifically on runtime detector selection instead of a fixed detector.

III. METHODOLOGY

A. Pipeline and detector pair

Each input video is processed frame by frame. Fig. 1 separates the deployed path from the offline evaluation path. The deployed path depends on the policy class. Scene-proxy policies first compute a low-cost visual signal, then run only the selected detector. Confidence-EMA is different: it first runs the fast detector as a watchdog, updates the confidence signal, and runs the accurate detector only if escalation is triggered. The selected detector output is the output used for that frame. Offline evaluation stores both fast and accurate outputs only to measure agreement, benefit-positive frames, and informed gain. The detector pair is YOLOv8 [2] at two sizes: nano (3.2M parameters) as the fast detector, and small or large as the accurate detector, depending on the experiment. Both detectors are trained per domain so that the accurate detector provides a stable reference.

B. Policy families

Eight policies are evaluated (Table I). Two static baselines (*fast-only* and *accurate-only*) bound the operating envelope. Four scene-proxy policies trigger before any detector inference and therefore pay a proxy cost on every frame. A confidence-watchdog policy, *confidence-EMA*, runs the fast detector first on every frame and escalates to the accurate detector when a fast-decaying EMA of $\bar{c}_t$ drops below a slow-decaying EMA. Concretely,

$$e_t^f = \beta_f \bar{c}_t + (1 - \beta_f) e_{t-1}^f, \quad (1)$$

$$e_t^s = \beta_s \bar{c}_t + (1 - \beta_s) e_{t-1}^s, \quad (2)$$

$$\delta_t = \max\left(0, \frac{e_t^s - e_t^f}{e_t^s}\right), \quad (3)$$

with $\beta_f=0.30$, $\beta_s=0.02$, and hysteresis on δ_t at $\delta_{\text{high}}=0.12$ and $\delta_{\text{low}}=0.04$. The signal fed to (1)–(3) is the *fast-detector* mean confidence, never the chosen-model confidence, so that the distribution feeding the EMA does not shift when the policy escalates. This fixed confidence source is important because neural-network confidence scores are not always calibrated probabilities [12]. A composite policy, *multi-proxy*, computes per-proxy EMA-deviations and thresholds a weighted sum with $w = (0.37, 0.29, 0.38, 0.28)$ over $\{L, H, \text{tenengrad}, \text{colour entropy}\}$. All policies share a per-frame stabiliser with a 10-frame minimum dwell time and a switch-rate cap of 12 per 100 frames.

TABLE I
EVALUATED POLICIES AND THEIR TRIGGER FAMILIES

Policy	Trigger / decision rule
fast-only	always fast (latency floor)
accurate-only	always accurate (reference)
entropy-only	$H \geq \text{rolling-median}_{200}(H)$
combined	weighted (L, H) , $C \geq 0.50$
combined-hyst.	weighted (L, H) , hysteresis $0.45 \rightarrow 0.55$
local-contrast hyst.	local contrast, hysteresis $0.45 \rightarrow 0.65$
confidence-EMA	$\delta_t \geq 0.12$, hysteresis to 0.04
multi-proxy	EMA-deviation composite, $C \geq 0.10$

C. Informed-gain metric

A policy that uses the accurate detector on, say, 70% of frames will appear to track the accurate detector closely whether or not its routing has frame-level signal. We separate routing skill from accurate-detector usage by correcting against a matched-random baseline. Let u be the fraction of frames routed to the accurate detector by the policy, and let A_{policy} be the per-frame agreement of the policy's selected outputs with the accurate detector. Define

$$\mathbb{E}[A]_{\text{rand}}(u) = (1-u) A_{\text{fast}} + u A_{\text{acc}}. \quad (4)$$

We report the informed gain

$$G_{\text{informed}} = A_{\text{policy}} - \mathbb{E}[A]_{\text{rand}}(u), \quad (5)$$

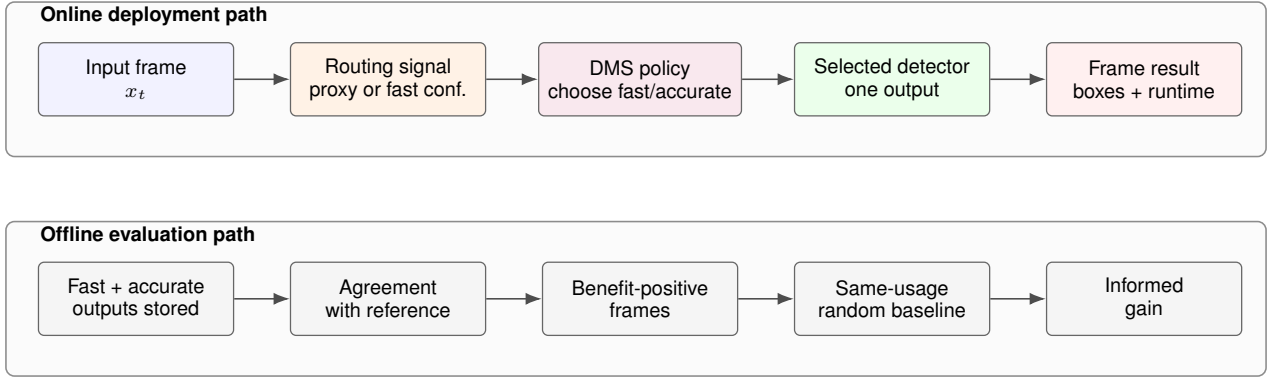
which is positive when the policy routes *better than chance* at the same accurate-detector usage. In practical terms, informed gain is the residual value of the trigger after the policy has already been charged for how often it uses the accurate detector. Small positive values therefore do not mean that new human-labelled defects have been proved; they mean that, at the same accurate usage, the trigger places the accurate detector on frames that better match the accurate-reference output than random switching would. Negative values indicate that the same accurate-detector budget is being spent on less useful frames than a usage-matched random policy. For the binary-classification control benchmark in Section V-F we use macro-F1 in place of agreement. Agreement is defined as the proportion of frames on which the policy-selected detector returns the same set of bounding boxes as the accurate detector under a 0.50 IoU match. Standard object-detection benchmarks evaluate against human labels [13]. This is an *accurate-reference* agreement, not human ground-truth mAP; the Anti-UAV benchmark provides the human ground truth (Section V-F).

D. Deployed runtime accounting

A trigger that selects the fast detector on most frames is not useful if the trigger itself costs more time than it saves. For scene-feature policies we bill

$$T_{\text{total}} = T_{\text{proxy}} + T_{\text{ctrl}} + T_{\text{selected}}, \quad (6)$$

where T_{proxy} is the per-frame proxy-extraction cost, T_{ctrl} is the controller logic cost, and T_{selected} is the latency of the



Offline evaluation is not part of deployed runtime; it is used only to validate routing decisions.

Fig. 1. Online and offline paths in the DMS benchmark. The deployed path emits one selected detector output per frame. The offline path stores both detector outputs only to compute agreement, benefit-positive frames, the same-usage random baseline, and informed gain.

selected detector. For confidence-EMA, the fast detector is the watchdog. Its runtime is therefore

$$T_{\text{total}} = T_{\text{fast}} + T_{\text{ctrl}} + \mathbf{1}_{\text{escalate}} T_{\text{accurate}}. \quad (7)$$

This accounting changes the picture significantly under hardware-dependent behaviour (Section V-D).

IV. EXPERIMENTAL SETUP

A. Data

The primary benchmark uses 14 UAV inspection videos: 9 in the glass insulator domain and 5 in the porcelain domain, comprising 79 429 unique frames. Two YOLO-family detector implementations are trained per domain at three sizes (n, s, l), yielding 12 detector checkpoints. The first is YOLOv8 [2]. The second is denoted YOLO26 because it is the second YOLO-family checkpoint set used in the training environment, with checkpoint names `yolo26n.pt`, `yolo26s.pt`, and `yolo26l.pt`. YOLO26 is used here only as a checkpoint-set label, not as a claim about a new detector architecture. Pair configurations include the common nano-to-large pair and several intermediate pairs. A secondary control benchmark uses 6 208 ground-truth images from the validation split of the Anti-UAV dataset [10]. This second benchmark is included precisely because it is a weak-opportunity regime: the fast and accurate detectors are within 0.004 macro-F1 of each other, and so the conditional claim predicts limited DMS upside there.

B. Hardware

Three hardware tiers are evaluated: (i) an AMD EPYC 8224P CPU (PyTorch CPU backend); (ii) an NVIDIA RTX 5090 GPU with CUDA execution; (iii) an NVIDIA Jetson Nano (Maxwell, 4 GB) with JetPack 4.6.1 and TensorRT 8.2 at FP16 [14], [15]. On the Jetson tier we measure detector-level per-inference energy from the POM_5V power rails and *project* loop-level energy from those measurements; a full-loop wattmeter trace remains as future work and is labelled as projected where it appears.

C. Sweep size

The full sweep produces 84 completed runs across 14 videos and six model/platform configurations. Each run generates per-frame CSV logs with frame-level choices, proxy values, EMA states, timing measurements, agreement scores, and benefit-positive flags. The final analysis aggregates these per-frame logs into 672 run-policy summary rows: 84 runs times eight policies. All summary rows are produced by a single aggregation script, so every figure and table in this paper is regenerable from the logged data.

V. RESULTS

A. Runtime–quality tradeoff (CPU)

Fig. 2 shows the CPU runtime–quality behaviour for the common n / l pair. Fast-only takes 86.1 ms per frame and reaches 0.636 agreement with the accurate detector; accurate-only takes 289.1 ms per frame and reaches 1.000 by definition. Confidence-EMA closes 43 % of the agreement gap at 66 % of accurate-only runtime, with accurate usage of 37.4 % and informed gain of +0.007 (Table II). Local-contrast hysteresis closes 55 % of the gap at 72 % of runtime, with informed gain +0.014. Combined-hysteresis sits between the two but is more variable across videos. Multi-proxy reaches 141.8 ms per frame on the same cut, but its informed gain is −0.007: the policy routes worse than a usage-matched random switcher. We report this as a negative result rather than tuning the policy until the number turns positive.

The informed-gain column is numerically small by construction: it is a residual over a same-usage random switcher, not the total distance between fast-only and accurate-only deployment, which on this cut is 0.364 agreement. An informed gain of +0.007 therefore means that confidence-EMA improves accurate-reference agreement by 0.7 percentage points beyond what its accurate-detector usage alone already buys. Over a one-hour 30FPS inspection stream that is on the order of 750 additional frame-level agreement outcomes against a random switcher held at the same compute budget — not

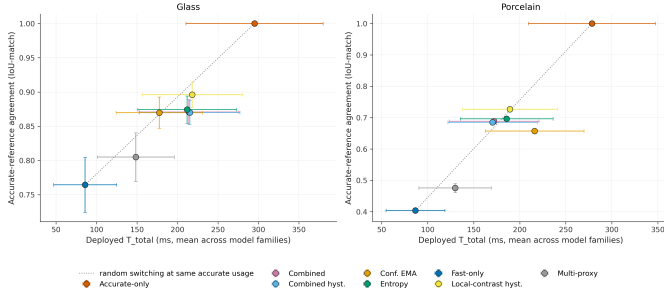


Fig. 2. CPU runtime–quality tradeoff on the common n/l pair. DMS policies occupy intermediate operating points between the two static baselines.

TABLE II
PER-POLICY RESULTS, CPU COMMON NANO-TO-LARGE PAIR

Policy	T_{total} (ms)	Agree.	u (%)	G
fast-only	86.1	0.636	0.0	—
accurate-only	289.1	1.000	100	—
entropy-only	202.5	0.811	49.7	+0.002
combined	199.1	0.806	47.6	+0.009
combined-hyst.	199.2	0.804	47.6	+0.008
local-contrast hyst.	207.8	0.836	52.1	+0.014
confidence-EMA	191.2	0.794	37.4	+0.007
multi-proxy	141.8	0.687	18.2	−0.007

u is accurate-detector usage; G is informed gain over the matched-random baseline.

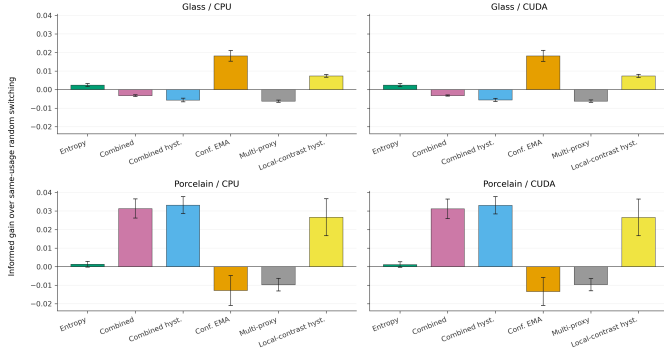


Fig. 3. Informed gain over a matched-random baseline, CPU common n/l pair. Multi-proxy crosses below zero, indicating routing worse than chance at the same accurate-detector usage.

750 additional detected defects, since the primary benchmark carries no dense human defect labels. The operational reading is that the gain is obtained at a fixed accurate-usage budget and accumulates over the long streams that inspection missions actually produce.

Fig. 3 summarises informed gain across evaluated policies on the CPU common cut. Positive bars indicate routing better than usage-matched random; the multi-proxy bar is the negative result.

B. Domain dependence

Pooling across glass and porcelain hides a large effect (Fig. 4). On glass, fast-only already reaches 0.77 agreement:

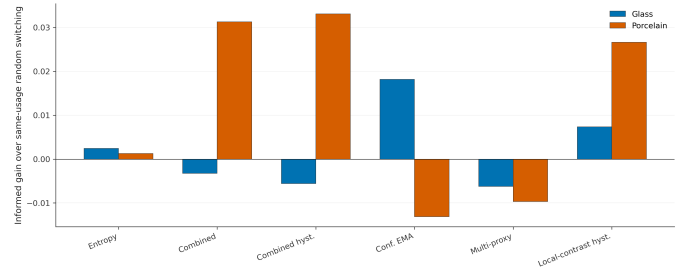


Fig. 4. Domain-split runtime–quality behaviour. Glass and porcelain do not share a single winning policy.

hard frames are relatively rare and the dynamic upside is modest. Confidence-EMA leads with pooled informed gain +0.019. On porcelain, fast-only drops to 0.39: hard frames are more common and the dynamic upside is correspondingly larger. Combined-hysteresis leads on porcelain with pooled informed gain +0.029 (and +0.033 on the CPU cut alone). The same confidence-EMA that wins glass produces *negative* informed gain on several porcelain cells, indicating that its threshold δ_{high} was effectively calibrated for the glass regime. A practical deployment therefore needs per-domain calibration of the watchdog threshold, not a single global setting.

C. Trigger validity

We treat the per-frame benefit-positive flag as a target and report trigger F1 against it. The benefit-positive rate across the sweep is approximately 21%, so a random F1 is around 0.21. The strongest dynamic policies (local-contrast hysteresis and confidence-EMA) reach trigger F1 around 0.25, measurably above random; entropy-only and combined sit closer to chance. Feature ranking is domain-sensitive: on glass the top trigger features are edge density, Laplacian variance, and Tenengrad; on porcelain Tenengrad and Laplacian remain strong, with hue variation prominent in some splits. We therefore claim feature-family association rather than a single universal scalar trigger.

D. Hardware-dependent behaviour

On the same CPU configuration the detector gap is $3.4\times$ (289.1 ms vs. 86.1 ms). On the RTX 5090 the gap shrinks to $1.3\times$ (9.9 ms vs. 7.6 ms): CUDA execution amortises the inference cost to the point at which the per-frame proxy-extraction cost dominates what is saved. Scene-proxy policies on the GPU therefore reach about $2.5\text{--}2.6\times$ the accurate-only runtime once the proxy and controller cost is billed. Only confidence-EMA stays near parity on the GPU (11.3 ms, $1.15\times$ accurate-only), because it does not extract scene proxies; even so, no policy saves runtime on this tier. The intended conclusion is not that DMS fails, but that it is hardware-conditional: the cost–benefit balance only favours switching where the detector gap stays large, which is the embedded regime examined next. Table III collects the three tiers side by side on the common nano-to-large pair. The detector gap is what moves — $3.4\times$ on CPU, $1.3\times$ on the GPU, $8.8\times$ on the

TABLE III
DEPLOYED PER-FRAME RUNTIME (MS) ACROSS HARDWARE TIERS

Platform	fast	acc.	gap	c.-EMA	ratio
CPU (EPYC 8224P)	86.1	289.1	3.4×	191.2	0.66×
GPU (RTX 5090)	7.6	9.9	1.3×	11.3	1.15×
Jetson Nano ^a	47.1	415.8	8.8×	163.9	0.39×

^aGlass domain, TensorRT FP16. Detector latencies measured; confidence-EMA entry projected (Section V-E).

Jetson — and the watchdog is economical exactly where that gap is wide. Ratios are confidence-EMA runtime relative to accurate-only, and the CPU and GPU rows pool both domains.

E. Embedded deployment (Jetson Nano)

Per-inference detector-level energy on the Jetson Nano POM_5V power rails was measured directly: 0.329J for YOLOv8 nano and 3.318J for YOLOv8 large (Fig. 5). The DMS figure on this tier is projected from those per-inference measurements rather than measured end to end. Under confidence-EMA the fast detector is the watchdog and runs on every frame, while the accurate detector runs only on the escalated fraction u , so the per-frame energy is approximated as

$$E_{\text{DMS}} = E_{\text{fast}} + uE_{\text{accurate}}. \quad (8)$$

With the measured glass escalation rate $u = 28.1\%$ this gives a lower-bound 1.261J per frame at a projected 6.1FPS, a 62.0% energy reduction against always running the accurate detector. Equation (8) is deliberately a lower bound: it charges only detector execution and excludes camera capture, logging, controller scheduling, and thermal throttling. The realised onboard value will therefore be higher, and will vary with input source, power mode, sustained thermal state, and mission-level scheduling. We consequently read the Jetson tier as evidence of embedded feasibility rather than as a full onboard UAV-loop power measurement; a wattmeter trace of the complete loop remains future work. YOLO26 was prepared for the Jetson stack but could not be deployed because the EfficientNMS_TRT plugin used by the export is not present in TensorRT 8.2 on JetPack 4.6.1. We treat this as a platform limitation, not a methodological one, and report YOLOv8 results only on this tier.

F. Weak-opportunity boundary (Anti-UAV)

The Anti-UAV control benchmark uses 6208 ground-truth images (Fig. 6). We report the YOLOv8 nano-to-large control slice. The benefit-positive rate is 1.1%, and the fast and accurate detectors land at 0.9772 and 0.9805 macro-F1 respectively ($\Delta = 0.0033$). The conditional claim predicts that DMS upside should be small in this regime, and it is. Confidence-EMA achieves 0.9791 macro-F1 at 16.4% accurate usage — close to accurate-only F1 while running noticeably faster, at 141 FPS vs. 134 FPS for accurate-only — but the informed F1 gain is only +0.0014. We treat this not as a counter-example to the

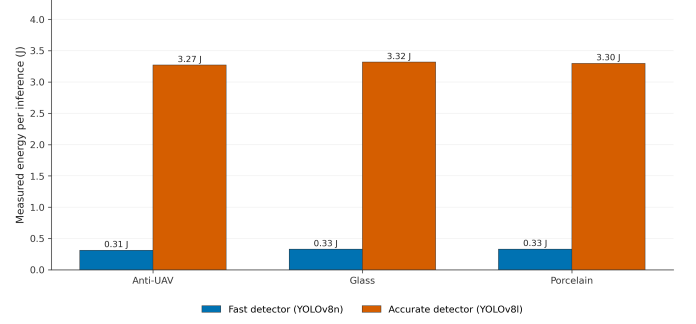


Fig. 5. Jetson Nano per-inference energy from the POM_5V power rails. The roughly tenfold gap between YOLOv8 nano and YOLOv8 large motivates DMS on this tier.

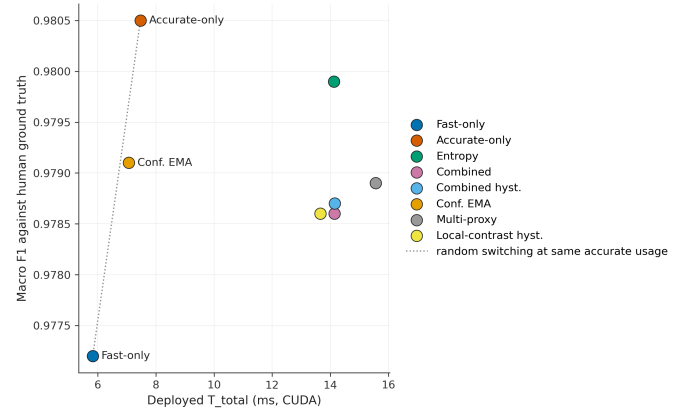


Fig. 6. Anti-UAV runtime–quality behaviour. The detector gap is small ($\Delta = 0.0033$ macro-F1), and DMS upside is limited ($G = +0.0014$), consistent with the conditional claim.

framework but as predicted behaviour: when fast and accurate are already close, switching has limited room to help.

VI. DISCUSSION

The three conditions under which DMS is a defensible improvement fall out of the methodology rather than being imposed on it. First, the accurate detector must add reference evidence the fast detector misses on a non-trivial fraction of frames; the Anti-UAV result shows what happens when this condition is weak. Second, the trigger must contain real frame-level signal at the chosen accurate-detector usage, otherwise informed gain is non-positive even on a fast platform; the multi-proxy negative result anchors this point and is reproducible. Third, the controller’s runtime cost must remain smaller than the inference cost it saves; the hardware-dependent result in Section V-D shows what happens when this condition fails.

Two specific methodological choices warrant a short defence. The first is the use of accurate-reference agreement rather than human ground truth on the primary benchmark; the second is the fast-detector confidence lock in (1)–(3). On the first, fully labelled UAV video for insulator defects at this scale is not available, and the Anti-UAV control benchmark

supplies the human ground truth that the primary benchmark does not. On the second, feeding chosen-model confidence into the EMAs would mix two different score distributions whenever the policy escalates, which would silently shift the trigger. Locking the EMA input to the fast-detector confidence keeps the trigger semantics fixed.

VII. CONCLUSION

We presented a runtime-aware framework for per-frame detector selection in UAV inspection, evaluated it across two domains and three hardware tiers, and proposed an informed-gain metric that separates routing skill from accurate-detector usage. The canonical confidence-watchdog policy closes a substantial part of the runtime-quality gap on CPU and on the embedded Jetson Nano, but only when (i) the accurate detector adds reference evidence, (ii) the trigger beats random switching at the same usage, and (iii) the controller cost remains smaller than the inference cost saved. A multi-proxy EMA-deviation composite is reported as a negative result with negative informed gain, and the hardware-dependent result shows that on a high-end GPU the scene-proxy family loses its economic basis. Future work should first replace the hand-tuned trigger thresholds with a learned trigger that fuses confidence and scene signals without target leakage. A full-loop wattmeter trace on the embedded tier is also needed to measure camera, logging, scheduling, and thermal overhead rather than projecting from detector-level energy alone. The confidence watchdog should be recalibrated under domain shift, especially for the porcelain regime where the current threshold is less reliable. Finally, the benchmark should be broadened beyond insulators to inspection targets such as conductors, towers, vegetation encroachment, and surface defects, and paired with densely labelled defect data where such labels are available, so that accurate-reference agreement can be compared directly with standard detection metrics on the same UAV sequences.

REFERENCES

- [1] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2016, pp. 779–788.
- [2] G. Jocher, A. Chaurasia, and J. Qiu, "Ultralytics YOLOv8," software and documentation, 2023. [Online]. Available: <https://github.com/ultralytics/ultralytics>
- [3] S. Xie *et al.*, "Real-time object detection from UAV inspection videos by combining YOLOv5s and DeepStream," *Sensors*, vol. 24, no. 12, art. 3862, 2024.
- [4] Y. Zhang, X. Yuan, and W. Li, "UAV-based inspection of power transmission lines: A survey," *IEEE Trans. Power Del.*, vol. 33, no. 6, pp. 3005–3014, Dec. 2018.
- [5] S. Teerapittayanon, B. McDanel, and H. T. Kung, "BranchyNet: Fast inference via early exiting from deep neural networks," in *Proc. Int. Conf. Pattern Recognit. (ICPR)*, 2016, pp. 2464–2469.
- [6] H. Cai, C. Gan, T. Wang, Z. Zhang, and S. Han, "Once-for-All: Train one network and specialize it for efficient deployment," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2020.
- [7] Y. Han, G. Huang, S. Song, L. Yang, H. Wang, and Y. Wang, "Dynamic neural networks: A survey," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 44, no. 11, pp. 7436–7456, 2022.
- [8] V. Sze, Y.-H. Chen, T.-J. Yang, and J. S. Emer, "Efficient processing of deep neural networks: A tutorial and survey," *Proc. IEEE*, vol. 105, no. 12, pp. 2295–2329, 2017.

- [9] A. Matathammal *et al.*, "EdgeMLBalancer: A self-adaptive approach for dynamic model switching on resource-constrained edge devices," arXiv:2502.06493, 2025.
- [10] N. Jiang *et al.*, "Anti-UAV: A large multi-modal benchmark for UAV tracking," arXiv:2101.08466, 2021.
- [11] B. Battseren, "Software architecture for real-time image analysis in autonomous MAV missions," Ph.D. dissertation, Technische Universität Chemnitz, 2024.
- [12] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, "On calibration of modern neural networks," in *Proc. Int. Conf. Mach. Learn. (ICML)*, 2017, pp. 1321–1330.
- [13] T.-Y. Lin *et al.*, "Microsoft COCO: Common objects in context," in *Proc. Eur. Conf. Comput. Vis. (ECCV)*, 2014, pp. 740–755.
- [14] NVIDIA Corporation, "TensorRT documentation," 2024. [Online]. Available: <https://docs.nvidia.com/deeplearning/tensorrt/>
- [15] NVIDIA Corporation, "Jetson Nano developer kit and JetPack 4.6.1 documentation," 2022. [Online]. Available: <https://developer.nvidia.com/embedded/jetpack>